Introduction to Docker

The "Works on My Machine" Problem

- You write a script — it runs perfectly on your laptop
- You send it to a classmate — it crashes on theirs
- Different **Python version**, different package versions, missing system libraries
- Deploying to a server? A different OS entirely

> *"It works on my machine" is not a deployment strategy*

What is Docker?

- A tool for packaging software into **containers**
- A container bundles **code + dependencies + runtime** into one portable unit
- Containers run the same way everywhere — laptop, cloud, classmate's machine
- A lightweight, reproducible box for your application

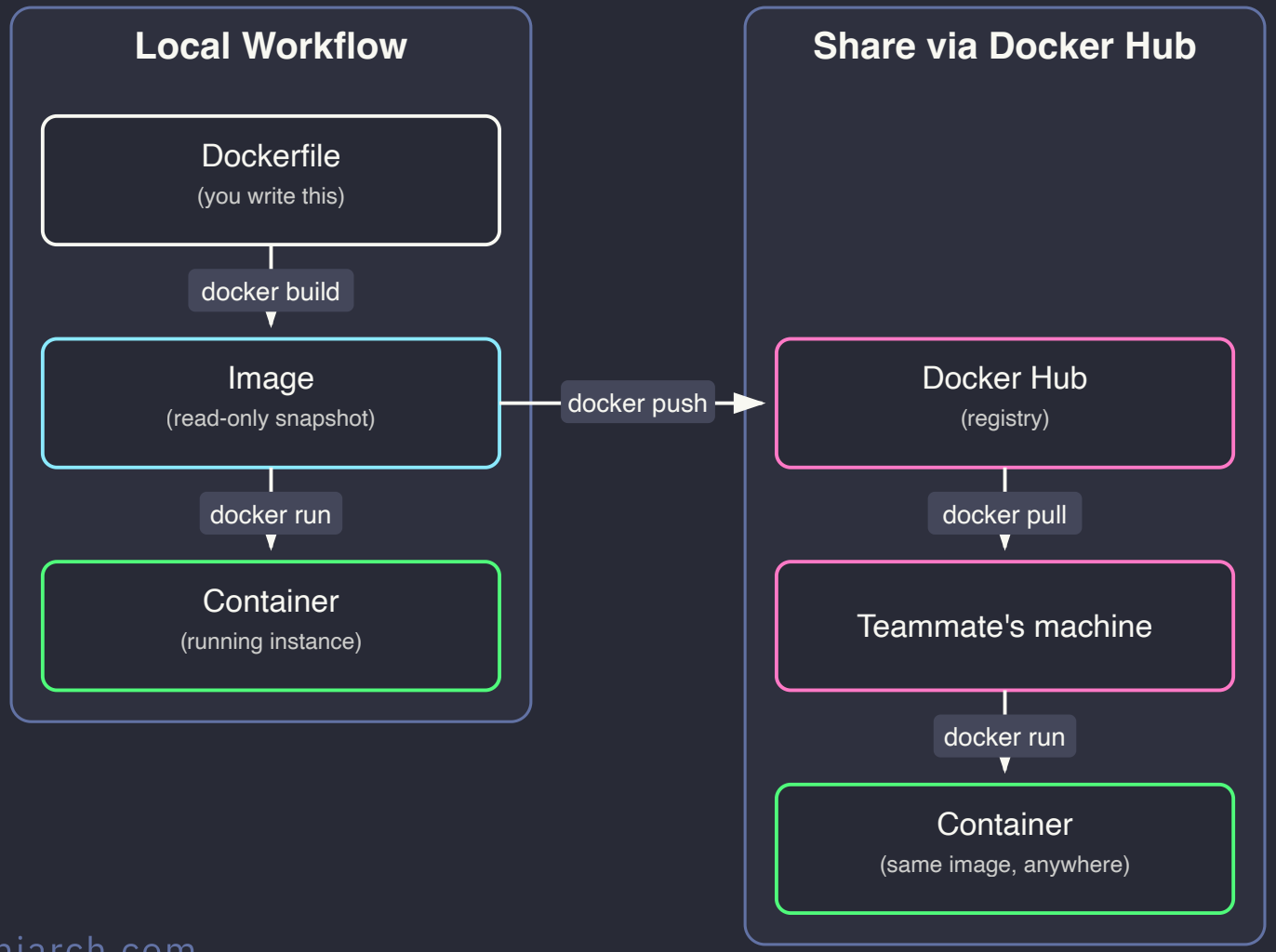
Without Docker	With Docker
"Install Python 3.11, then pip install..."	`docker run my-app`
Breaks across OS / environments	Same behaviour everywhere
Manual setup on every machine	One image, runs anywhere

Key Concepts

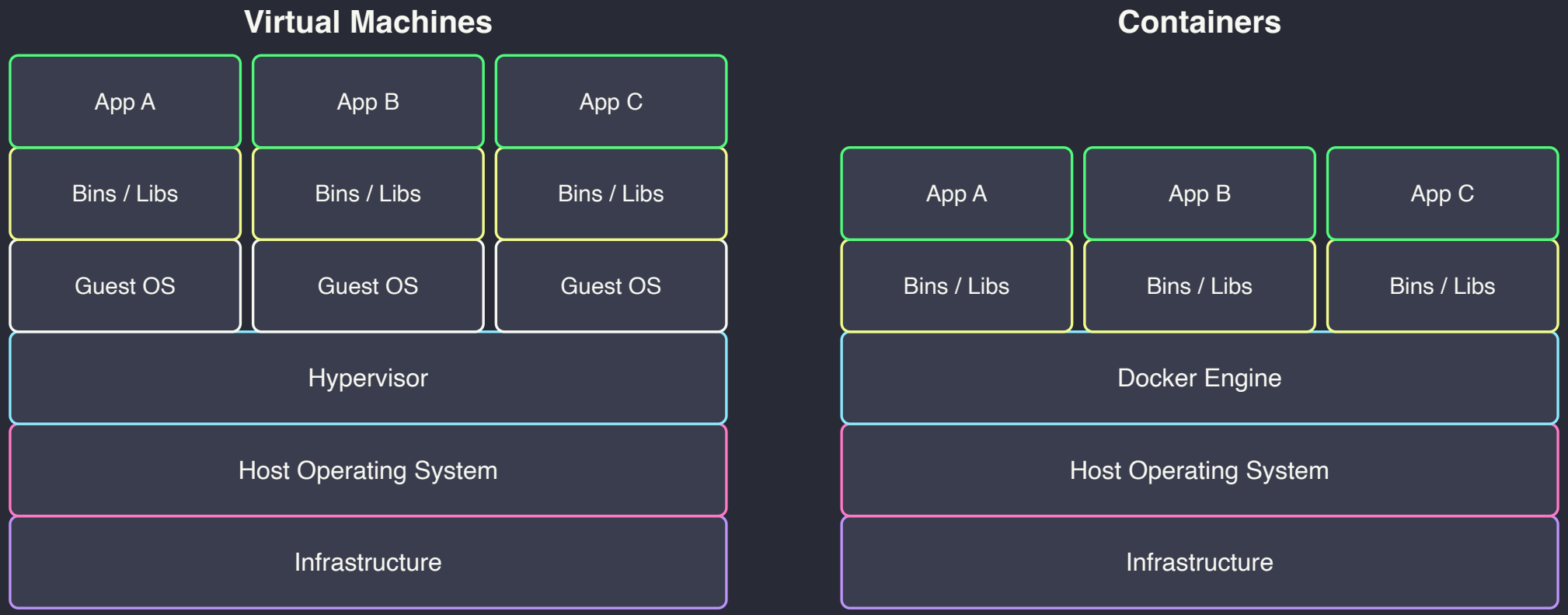
- **Dockerfile** — a text recipe describing how to build your environment
- **Image** — the built snapshot created from a Dockerfile (read-only)
- **Container** — a running instance of an image (many containers, one image)
- **Docker Hub** — a public registry of pre-built images (like PyPI for containers)
- **Layer** — each Dockerfile instruction adds a cached layer

> *Image is to Container what a **Class** is to an **Object** in Python*

Docker Workflow



Docker vs Virtual Machine



> *Containers are not VMs — they are isolated **processes**, not emulated machines*

Installing Docker

1. Download **Docker Desktop** from `docker.com/get-started`
   - Available for macOS, Windows, Linux
2. Follow the installer — Docker Desktop includes the CLI and GUI
3. Verify the install:

```
docker --version
# Docker version 27.x.x

docker run hello-world
# Should print: "Hello from Docker!"
```

> *On Linux: install Docker Engine directly — no Desktop needed*

Essential Docker Commands

```
# Build an image
docker build -t my-app .

# List local images
docker images

# Run a container
docker run my-app

# Interactive shell in container
docker run -it my-app bash
```

```
# List running containers
docker ps

# Show all containers (incl. stopped)
docker ps -a

# Stop / remove a container
docker stop <container_id>
docker rm <container_id>

# Pull image from Docker Hub
docker pull python:3.11-slim
```

Writing a Dockerfile

```
# 1. Base image — Python 3.11 without unnecessary extras
FROM python:3.11-slim

# 2. Set working directory inside the container
WORKDIR /app

# 3. Copy and install dependencies FIRST (layer cache trick)
COPY requirements.txt .
RUN pip install -r requirements.txt

# 4. Copy the application code
COPY app.py .

# 5. Default command to run when the container starts
CMD ["python", "app.py"]
```

Demo: Your First Container

Project files:

```
docker_demo/
├── Dockerfile
├── requirements.txt
└── app.py
```

- `Dockerfile` — the build recipe
- `requirements.txt` — one dependency: `pyfiglet`
- `app.py` — prints a banner + the Python version it's running on

`app.py` :

```python
import platform
import pyfiglet

banner = pyfiglet.figlet_format("Hello Docker")
print(banner)
print(f"Python version : {platform.python_version()}")
print(f"Platform       : {platform.system()}")
print("Running inside a container!")
```

Running the Demo

Commands:

```
# Navigate to the demo folder
cd docker_demo

# Build the image (~10s first time)
docker build -t docker-demo .

# Run the container
docker run --rm docker-demo
```

Expected output:

```
 _   _      _ _
| | | | ___| | | ___
| |_| |/ _ \ | |/ _ \
|  _  |  __/ | | (_) |
|_| |_|\___|_|_|\___/

Python version : 3.11.x
Platform       : Linux
Running inside a container!
```

Useful Patterns

- **Mount a local folder** to read/write files without rebuilding:

```
docker run -v $(pwd)/data:/app/data docker-demo
```

- **Set environment variables** (API keys, config):

```
docker run -e API_KEY=abc123 my-app
```

- **Expose a port** for a web API (e.g., Flask/FastAPI):

```
docker run -p 8000:8000 my-api
```

Where Docker Fits

Stage	Docker Use
Development	Reproducible environment across the whole team
Testing / CI	Run tests in the same container that ships to prod
Sharing	`docker push` an image instead of a setup guide
Deployment	Package app + dependencies as one image
Cloud	AWS, Azure, GCP all run and scale container images

> *Learn Docker once — the skill transfers to every cloud platform*

Common Pitfalls

- **Forgetting `-t` on build** — you get an unnamed image, hard to reference later
- **Putting `COPY . .` before `pip install`** — busts the layer cache on every code change
- **Not using `.dockerignore`** — copies `.git/`, `__pycache__/`, `.venv/` into the image, bloating it
- **Hardcoding secrets in the Dockerfile** — use `-e` environment variables instead
- **Forgetting the container is isolated** — files written inside it vanish unless a volume is mounted

References

Topic	Source
Docker Get Started guide	Docker Inc. (2026). Get Started with Docker. Docker Docs.
Dockerfile reference	Docker Inc. (2026). Dockerfile reference. Docker Docs.
Docker Hub	Docker Inc. (2026). Docker Hub.
Official Python images	Docker Inc. (2026). python. Docker Hub.
`.dockerignore` **reference**	Docker Inc. (2026). .dockerignore file. Docker Docs.

> *Start with the official Get Started guide — it covers everything in an interactive, no-install browser tutorial*

Recap & Practice

- **Docker** packages code + dependencies into portable containers
- **Dockerfile → Image → Container** is the core workflow
- Containers are lighter and faster than VMs — same code, everywhere
- Three commands to remember: `docker build`, `docker run`, `docker ps`
- **Before your next project: containerise it instead of writing a setup README**

> *Ship your environment, not just your code*